

EKS Production Stack

Comprehensive DevOps Exercises

- ◆ Helm Charts for Every Service
 - ◆ Helmfile Orchestration
- ◆ Node Affinity & Taints/Tolerations
- ◆ Prometheus + Grafana Monitoring
 - ◆ HPA & VPA Autoscaling
 - ◆ Database Backups to S3
- ◆ ExternalDNS + ExternalSecrets
- ◆ Karpenter / Cluster Autoscaler

Intermediate Level — Requirements-Based Exercises

Students choose their own application stack (frontend + backend + database)

Table of Contents

Section 1 — Architecture Overview

Section 2 — Helm Charts — One Per Service (*Frontend, Backend, Database*)

Section 3 — What Is Helmfile? + Helmfile Setup (*Declarative multi-chart management*)

Section 4 — Node Separation — Affinity & Taints (*App nodes vs DB nodes*)

Section 5 — Monitoring — Prometheus & Grafana (*kube-prometheus-stack + ServiceMonitors*)

Section 6 — HPA & VPA — Autoscaling (*Horizontal & Vertical Pod Autoscalers*)

Section 7 — Database Backups (*CronJob to S3 with lifecycle policies*)

Section 8 — ExternalDNS (*Automatic Route53 DNS management*)

Section 9 — ExternalSecrets (*AWS Secrets Manager to K8s Secrets*)

Section 10 — Karpenter / Cluster Autoscaler (*Intelligent node provisioning*)

Section 11 — Final Integration Exercise (*Deploy everything end-to-end*)

Section A — Appendix — Useful Commands

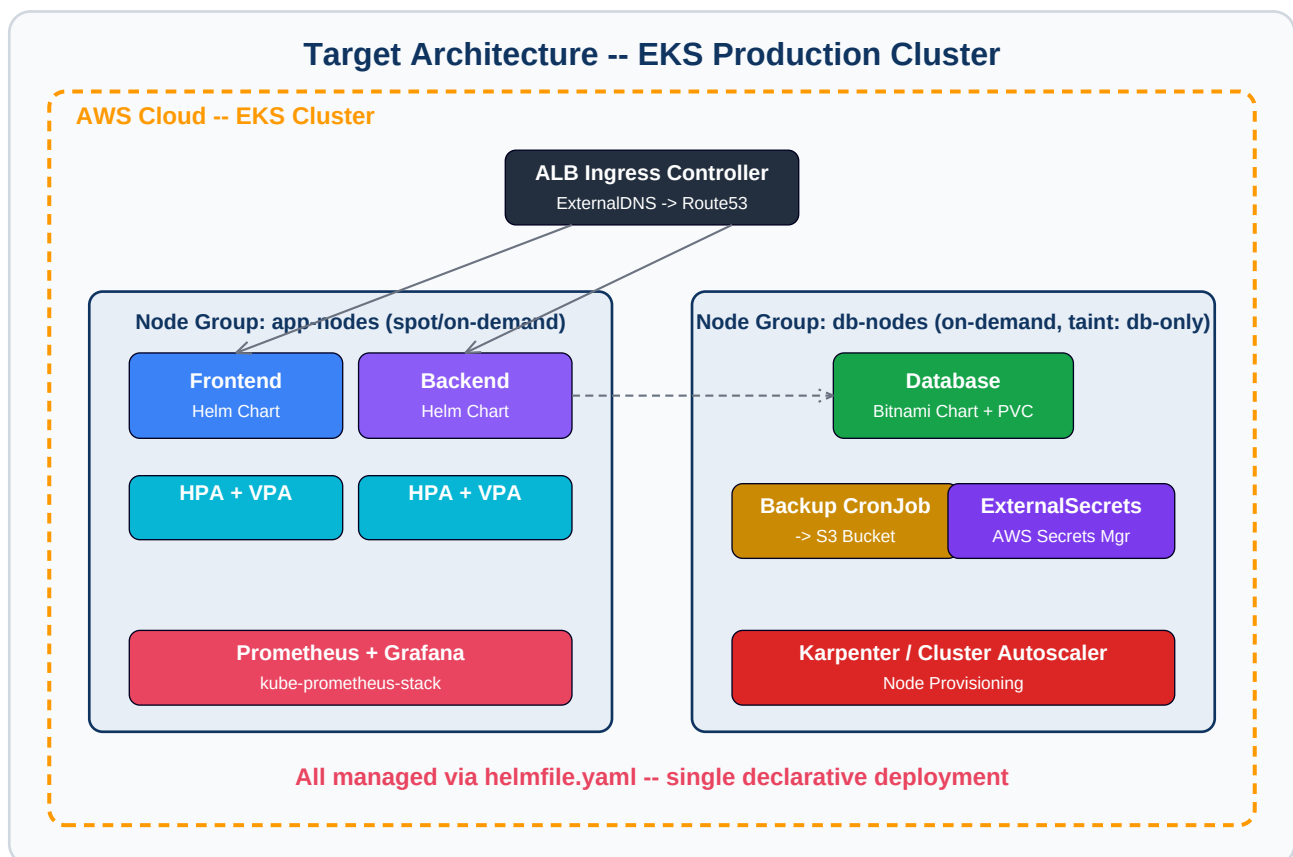
1

Architecture Overview

The big picture of what you're building

In this project you will build a production-grade EKS cluster that runs a three-tier application (frontend + backend + database), fully managed through Helm charts and orchestrated by Helmfile. The architecture enforces node separation, autoscaling at both pod and node level, automated monitoring, secrets management, DNS automation, and database backups.

Target Architecture Diagram



Key Design Decisions

- **One Helm chart per service** — frontend and backend get custom charts; the database uses a production-ready Bitnami chart as a dependency (wrapper chart pattern).
- **Helmfile as the orchestrator** — a single helmfile.yaml declares all releases, environments, and dependencies.
- **Node separation** — frontend and backend share app-nodes; the database runs on dedicated db-nodes using taints and tolerations.
- **Monitoring per service** — each chart includes a ServiceMonitor that Prometheus auto-discovers.

- **HPA for stateless, VPA for tuning** — horizontal scaling for front/back, vertical resource recommendations for all.
- **Secrets never in Git** — ExternalSecrets Operator syncs from AWS Secrets Manager.
- **DNS automation** — ExternalDNS creates Route53 records from Ingress annotations.
- **Smart node scaling** — Karpenter provisions right-sized nodes on demand.

Your Stack, Your Choice

You choose your own application stack. Examples: React + Express + PostgreSQL, Nginx + FastAPI + MySQL, Vue + Go + MongoDB. The exercises focus on infrastructure, not application code. Use any containerized app you already have or create minimal ones.

2

Helm Charts — One Per Service

Frontend, Backend, Database

Each service gets its own Helm chart. For the frontend and backend you will write custom charts from scratch with their own `Chart.yaml`, `values.yaml`, and `templates/` directory. For the database, you will use a **production-ready Bitnami chart** as a subchart dependency — this is how real teams deploy databases in production. You create a thin wrapper chart that declares the Bitnami chart in `Chart.yaml` dependencies and overrides its values.

Expected Directory Structure

Your project should have a `charts/` directory containing three subdirectories: `frontend/`, `backend/`, and `database/`. The frontend and backend charts each need a `Chart.yaml`, `values.yaml`, and a `templates/` directory containing templates for: Deployment, Service, Ingress, HPA, and ServiceMonitor. The database chart is a wrapper — its `Chart.yaml` declares the Bitnami chart as a dependency, and its `templates/` directory only contains your custom additions: a backup CronJob and optionally a ServiceMonitor if the Bitnami built-in one isn't sufficient.

Exercise 2.1: Create the Frontend Helm Chart

- ▶ Create a Helm chart named **frontend** under `charts/frontend/`
- ▶ The Deployment should be parameterized: image, tag, replicas, resource requests/limits all from `values.yaml`
- ▶ Include a **Service** (ClusterIP) and an **Ingress** resource with annotations for ALB Ingress Controller
- ▶ Add **nodeAffinity** or **nodeSelector** so the frontend runs only on nodes labeled `role=app`
- ▶ Add a **readinessProbe** and **livenessProbe**
- ▶ Include a **ServiceMonitor** template (can be toggled on/off via values)
- ▶ Validate: `helm template` should render without errors

Hint: Use `helm create frontend` as a starting point, then strip what you don't need.

Exercise 2.2: Create the Backend Helm Chart

- ▶ Create a Helm chart named **backend** under `charts/backend/`
- ▶ Same structure as frontend: Deployment, Service (ClusterIP), Ingress, ServiceMonitor
- ▶ The backend needs **environment variables** for database connection — use a ConfigMap for non-sensitive values and reference a **Secret** (will come from ExternalSecrets later) for credentials
- ▶ Add **nodeAffinity** targeting `role=app` nodes (same as frontend)
- ▶ Include readiness/liveness probes and resource requests/limits
- ▶ Validate: `helm template` renders cleanly

Hint: For the DB Secret reference, use `valueFrom.secretKeyRef` — the actual Secret will be created by ExternalSecrets Operator in Section 9.

Exercise 2.3: Create the Database Wrapper Chart (Bitnami)

Instead of writing a StatefulSet from scratch, you will use a **Bitnami chart** as a subchart dependency. This is production best practice — Bitnami charts handle StatefulSets, PVCs, health checks, metrics exporters, and dozens of edge cases you would otherwise need to solve yourself. You create a thin "wrapper" chart that declares the dependency and overrides values.

Why Use Bitnami Instead of Writing Your Own?

- **Battle-tested** — Bitnami charts are used in thousands of production clusters, continuously updated
- **Built-in metrics exporter** — `postgres-exporter` / `mysqld-exporter` included as a sidecar, just enable it
- **Automatic PVC management** — StatefulSet with `volumeClaimTemplates` configured correctly
- **Security hardened** — non-root containers, `securityContext`, `NetworkPolicies` out of the box
- **HA support** — replication, read replicas, and failover configurable via values

Wrapper Chart Pattern

In your `charts/database/Chart.yaml`, declare the Bitnami chart for your chosen database as a dependency (using the `dependencies` field). Pin it to a major version. Then run `helm dependency update` to pull the subchart. All Bitnami value overrides go under the dependency name key in your `values.yaml` (e.g., under `postgresql:`).

- ▶ Create a wrapper chart named **database** under `charts/database/`
- ▶ In `Chart.yaml`, declare the Bitnami chart for your chosen DB (`postgresql`, `mysql`, `mongodb`, `mariadb`, etc.) as a dependency
- ▶ Run `helm dependency update charts/database/` to pull the subchart
- ▶ In `values.yaml`, override the Bitnami defaults:
 - ▶ • Set **`auth.postgresPassword`** (or equivalent) — later this will be replaced by `ExternalSecrets`
 - ▶ • Enable **`metrics.enabled: true`** to activate the built-in exporter sidecar
 - ▶ • Configure **`primary.persistence`** with `storageClass: gp3` and appropriate size
 - ▶ • Set **`primary.resources`** (requests and limits)
- ▶ Add **`tolerations`** for `taint workload=database:NoSchedule` and **`nodeAffinity`** for `role=db` nodes via Bitnami's values (`primary.tolerations` and `primary.nodeAffinityPreset` or `primary.affinity`)
- ▶ Add a custom **backup CronJob** template in `templates/` (details in Section 7)
- ▶ Validate: `helm template charts/database/` should render the full StatefulSet, Service, and your custom templates

Hint: Bitnami values are nested under the dependency name. If your dependency is called postgresql, set values under postgresql: in your values.yaml. Check `helm show values bitnami/postgresql` for all available options.

Best Practice

Always set resource **requests** on every container. Without them, HPA cannot calculate utilization, VPA has no baseline to optimize, and the scheduler makes poor placement decisions. Start with conservative requests and let VPA recommend adjustments. For the Bitnami DB chart, set these under `primary.resources` in your wrapper values.

3

What Is Helmfile?

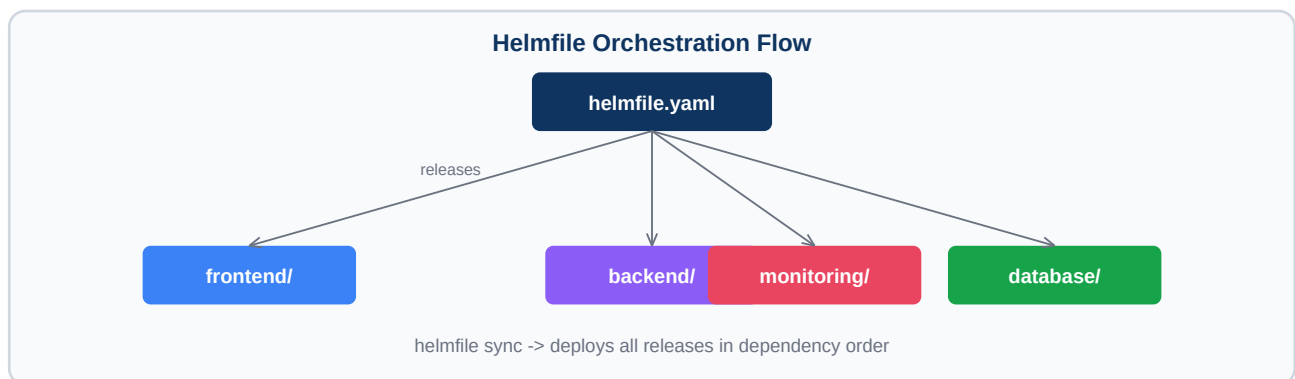
Declarative multi-chart management

What Is Helmfile?

Helmfile is a declarative tool that manages multiple Helm releases as a single unit. Think of it this way: if Helm is like `apt install nginx` (installing one package), then Helmfile is like a `requirements.txt` or `docker-compose.yaml` — it defines ALL the packages (Helm releases) your cluster needs in one file, and deploys them together.

Why Helmfile Instead of Raw Helm?

- **Single source of truth** — one `helmfile.yaml` declares every release in your cluster
- **Dependency ordering** — database deploys before backend, backend before frontend
- **Environment overrides** — different values for dev/staging/prod without duplicating charts
- **Atomic operations** — `helmfile sync` installs/upgrades everything; `helmfile destroy` tears it all down
- **DRY configuration** — shared values, templates, and repository definitions
- **Diff before apply** — `helmfile diff` shows what will change before you commit



Helmfile vs Helm — Comparison

Feature	helm install/upgrade	helmfile sync
Deploy 5 charts	5 separate commands	1 command
Dependencies	Manual ordering	needs: [release-name]
Environments	helm -f prod.yaml ...	Built-in env support
Diff changes	Not built-in	helmfile diff
Teardown all	5 helm uninstall cmds	helmfile destroy

Git-friendly	Scripts needed	Single YAML file
--------------	----------------	------------------

Core Helmfile Concepts

A `helmfile.yaml` has two main sections: **repositories** (where to find charts) and **releases** (what to deploy). Each release specifies a chart source (local path or remote repo), a target namespace, optional `needs`: dependencies (other releases that must deploy first), and a reference to values files. Helmfile resolves the dependency graph and deploys releases in the correct order.

Exercise 3.1: Install Helmfile and Initialize the Project

- ▶ Install **helmfile** on your machine (binary from GitHub releases or via package manager)
- ▶ Install the **helm-diff** plugin: `helm plugin install https://github.com/databus23/helm-diff`
- ▶ Create the project root with: `helmfile.yaml` at the root, a `charts/` directory containing `frontend/`, `backend/`, and `database/`, and a `values/` directory containing override files for each release (`frontend`, `backend`, `database`, `monitoring`)
- ▶ Run `helmfile lint` — it should pass with no errors

Exercise 3.2: Build the Complete `helmfile.yaml`

- ▶ Declare all **repositories** you need (`prometheus-community`, `external-secrets`, etc.)
- ▶ Define releases for: **frontend**, **backend**, **database**, **kube-prometheus-stack**, **external-secrets**, **external-dns**
- ▶ Set correct **needs**: dependencies — `database` first, then `backend`, then `frontend`
- ▶ Each release should reference values from the `values/` directory
- ▶ Monitoring and infra releases (`external-secrets`, `external-dns`) should deploy to their own namespaces
- ▶ Run `helmfile diff` to preview what would be deployed
- ▶ Run `helmfile sync` to deploy everything

Hint: Use `helmfile --environment staging sync` to test environment overrides later.

Warning

`helmfile sync` will install new releases AND upgrade existing ones. Always run `helmfile diff` first to review changes. In CI/CD, use `helmfile apply` which combines `diff` + `sync`.

4

Node Separation

App nodes vs Database nodes

In production, you typically want your database on dedicated nodes — isolated from application traffic, using on-demand instances (not spot), and often with higher memory. Kubernetes provides three mechanisms to achieve this: **nodeSelector**, **nodeAffinity**, and **taints/tolerations**.

How It Works Together

- **Labels** on nodes: `role=app` and `role=db`
- **Taint** on db-nodes: `workload=database:NoSchedule` — prevents non-DB pods from landing there
- **Toleration** on DB pods: allows them to be scheduled on tainted nodes
- **nodeAffinity** on DB pods: attracts them specifically to `role=db` nodes
- **nodeAffinity** on app pods: attracts frontend/backend to `role=app` nodes

Exercise 4.1: Configure EKS Node Groups

- ▶ Create (or verify) two EKS managed node groups:
- ▶ • **app-nodes**: label `role=app`, instance type of your choice, min 2 / max 5
- ▶ • **db-nodes**: label `role=db`, instance type with higher memory, min 1 / max 2, **on-demand only**
- ▶ Apply taint `workload=database:NoSchedule` to db-nodes
- ▶ Verify labels: `kubectl get nodes --show-labels`
- ▶ Verify taint: `kubectl describe node` should show the taint

Exercise 4.2: Update Charts with Node Placement Rules

- ▶ In the **frontend** and **backend** charts, add `nodeAffinity` (`preferredDuringScheduling`) for `role=app`
- ▶ In the **database** chart, add:
 - ▶ • `nodeAffinity` (`requiredDuringScheduling`) for `role=db`
 - ▶ • `tolerations` for `workload=database:NoSchedule`
- ▶ Make all affinity/toleration values configurable through `values.yaml`
- ▶ Deploy and verify: `kubectl get pods -o wide` should show app pods on app-nodes and DB pods on db-nodes

Hint: Use `requiredDuringSchedulingIgnoredDuringExecution` for the DB (hard requirement) and `preferredDuringSchedulingIgnoredDuringExecution` for apps (soft preference).

Best Practice

Use **required** affinity for databases (must be on db-nodes) and **preferred** for app pods (should be on app-nodes, but can land elsewhere if needed). This prevents scheduling failures when app-nodes are temporarily full while Karpenter provisions more.

5

Monitoring

Prometheus & Grafana with kube-prometheus-stack

The **kube-prometheus-stack** Helm chart deploys Prometheus, Grafana, Alertmanager, and a set of default dashboards and recording rules. It uses the **Prometheus Operator** pattern: instead of editing Prometheus config files, you create ServiceMonitor CRDs that Prometheus automatically discovers and scrapes.

Exercise 5.1: Deploy kube-prometheus-stack via Helmfile

- ▶ Add the **kube-prometheus-stack** release to your `helmfile.yaml`
- ▶ Deploy it to namespace `monitoring`
- ▶ In your values override:
 - ▶ • Enable **persistent storage** for Prometheus (at least 10Gi)
 - ▶ • Set Prometheus **retention** to 7 days
 - ▶ • Enable **Grafana** with an admin password from a Secret (not plaintext in values)
 - ▶ • Enable `serviceMonitorSelectorNilUsesHelmValues: false` so Prometheus discovers ServiceMonitors from all namespaces
- ▶ Verify: `kubectl get pods -n monitoring` — Prometheus, Grafana, and Alertmanager should be running
- ▶ Port-forward Grafana and confirm you can log in

Exercise 5.2: Create ServiceMonitors for Each Service

- ▶ In the **frontend** chart, create a ServiceMonitor that scrapes the frontend's metrics endpoint
- ▶ In the **backend** chart, create a ServiceMonitor for backend metrics
- ▶ In the **database** chart, create a ServiceMonitor for database metrics
- ▶ Each ServiceMonitor should:
 - ▶ • Have a `matchLabels` selector targeting its service
 - ▶ • Specify the correct port and path for metrics
 - ▶ • Be toggleable via `values.yaml` (e.g., `metrics.enabled: true`)
- ▶ Verify: in Prometheus UI (port-forward 9090), go to Status -> Targets and confirm all three ServiceMonitors appear

Hint: For databases using Bitnami charts, just set `metrics.enabled: true` in your values — the exporter sidecar is built in. For custom apps, your frontend/backend need to expose a `/metrics` endpoint (use `prom-client` for Node.js, `prometheus_client` for Python, etc.).

Exercise 5.3: Build Grafana Dashboards

- ▶ Import or create a Grafana dashboard for each service:
 - ▶ • **Frontend**: request rate, error rate, latency (p50/p95/p99)
 - ▶ • **Backend**: request rate, error rate, latency, active connections
 - ▶ • **Database**: connections, query rate, replication lag (if applicable), disk usage

- ▶ Create at least one **PrometheusRule** alert (e.g., high error rate > 5% for 5 minutes)
- ▶ Bonus: provision dashboards via ConfigMap so they survive Grafana restarts

Hint: Search Grafana.com for community dashboards using IDs — e.g., dashboard 1860 for node metrics.

Common Gotcha

Always set `serviceMonitorSelectorNilUsesHelmValues: false` in the `kube-prometheus-stack` values. Without this, Prometheus only discovers ServiceMonitors with specific labels matching its own Helm release — and your app ServiceMonitors will be silently ignored.

6

HPA & VPA — Autoscaling

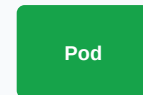
Horizontal & Vertical Pod Autoscalers

HPA vs VPA -- Scaling Dimensions

HPA -- Horizontal



VPA -- Vertical



Use HPA for stateless (front/back), VPA for resource tuning. Avoid both on same metric.

HPA — Horizontal Pod Autoscaler

HPA scales the **number of pods** based on observed metrics (CPU, memory, or custom metrics). It's the right choice for stateless services like your frontend and backend — when traffic increases, add more pods; when it drops, scale down.

VPA — Vertical Pod Autoscaler

VPA adjusts the **resource requests and limits** of existing pods. It observes actual resource usage over time and recommends (or automatically applies) right-sized CPU/memory requests. This is especially useful for databases and services where horizontal scaling isn't straightforward.

Warning — HPA + VPA Conflict

Do NOT use HPA and VPA on the **same metric** for the same deployment. If HPA scales on CPU and VPA also adjusts CPU requests, they will fight each other. Common pattern: HPA on CPU for scaling, VPA in Off or Initial mode for recommendations only.

Exercise 6.1: Deploy the Metrics Server

- Verify the **metrics-server** is running in your cluster (EKS often includes it)
- If not present, deploy it via Helmfile (add it as a release)
- Verify: `kubectl top nodes` and `kubectl top pods` should return data

Hint: On EKS, metrics-server is usually pre-installed. Check `kubectl get deployment metrics-server -n kube-system`.

Exercise 6.2: Configure HPA for Frontend and Backend

- In the **frontend** chart, create an HPA resource:
- • Min replicas: 2, Max replicas: 10

- ▶ • Scale on CPU utilization > 70%
- ▶ • Add a memory-based target as well (e.g., 80%)
- ▶ Do the same for the **backend** chart
- ▶ Make all HPA parameters configurable via `values.yaml`
- ▶ Include behavior rules to prevent flapping: scale-up cooldown 60s, scale-down cooldown 300s
- ▶ Load test (use `kubectl run` with a curl loop or a tool like hey) and verify pods scale up
- ▶ Verify: `kubectl get hpa` should show current/target metrics and replica counts

Exercise 6.3: Deploy and Configure VPA

- ▶ Install the **Vertical Pod Autoscaler** in your cluster (add to Helmfile or deploy from official manifests)
- ▶ Create a VPA resource for the **database** StatefulSet in `updateMode: "Off"` (recommendation only)
- ▶ Create a VPA resource for **frontend** and **backend** in `updateMode: "Off"` (since HPA is active)
- ▶ After running your app for at least 10 minutes, check: `kubectl describe vpa` for recommended values
- ▶ Compare VPA recommendations to your initial resource requests — adjust if significantly different

Best Practice

In production, use VPA in `Off` mode alongside HPA. Read the VPA recommendations periodically and manually update your `values.yaml` resource requests. This gives you right-sized pods without the risk of automatic restarts.

7

Database Backups

CronJob to S3 with lifecycle policies

Why CronJob + S3? (Best Practice Rationale)

There are several approaches to database backups in Kubernetes: **Velero** (full cluster/volume snapshots), **EBS snapshots** (AWS-native volume snapshots), and **CronJob with dump + S3**. For database-specific backups, the CronJob approach is the best practice for these reasons:

- **Application-consistent** — `pg_dump` / `mysqldump` produce logically consistent backups, not filesystem snapshots that may capture a write in progress
- **Portable** — SQL dumps can be restored to any instance, any cloud, any version — not tied to EBS or AWS
- **Granular** — you can back up specific databases or tables, not just entire volumes
- **Verifiable** — you can easily test restores by importing the dump into a fresh instance
- **Cost-effective** — S3 with lifecycle policies (move to Glacier after 30 days, delete after 90) is far cheaper than EBS snapshots
- **S3 durability** — 99.999999999% (11 nines) durability, cross-region replication available

Velero is complementary — use it for full cluster disaster recovery, but rely on application-level dumps for your database backup strategy.

Database Backup Strategy



K8s CronJob -> dump -> gzip -> aws s3 cp -> S3 with lifecycle policy

Exercise 7.1: Create the Backup CronJob

- In the **database** chart, create a CronJob template that:
 - • Runs on a schedule (e.g., every 6 hours: `0 */6 * * *`)
 - • Uses an init container or main container with your DB client tool installed
 - • Executes the appropriate dump command (`pg_dump`, `mysqldump`, `mongodump`, etc.)
 - • Compresses the output with `gzip`
 - • Uploads to S3 with a timestamped filename: `s3://bucket/backups/db-$(date +%Y%m%d-%H%M%S).sql.gz`

- ▶ • Uses a **ServiceAccount** with IRSA (IAM Role for Service Account) for S3 access — no hardcoded AWS keys
- ▶ Make schedule, bucket name, and retention configurable in `values.yaml`
- ▶ Set `concurrencyPolicy: Forbid` and `successfulJobsHistoryLimit: 3`

Exercise 7.2: Configure S3 Lifecycle Policy

- ▶ Create an S3 bucket for backups with:
 - ▶ • **Versioning** enabled
 - ▶ • **Encryption** (SSE-S3 or SSE-KMS)
 - ▶ • **Lifecycle policy**:
 - ▶ — Move to Standard-IA after 30 days
 - ▶ — Move to Glacier after 60 days
 - ▶ — Delete after 90 days
- ▶ Create an **IAM role** with S3 PutObject + GetObject permissions, scoped to the backup bucket
- ▶ Associate the IAM role with a Kubernetes ServiceAccount using **IRSA**

Exercise 7.3: Test Backup and Restore

- ▶ Trigger a manual backup: `kubectl create job --from=cronjob/ manual-backup-test`
- ▶ Verify the backup file exists in S3
- ▶ Restore the backup to a **fresh database instance** (spin up a temporary pod)
- ▶ Verify data integrity after restore
- ▶ Document the restore procedure in a README

Golden Rule of Backups

Always test your restores. A backup that has never been restored is not a backup — it's a hope. Add a periodic restore test to a non-production environment as part of your pipeline.

ExternalDNS watches your Kubernetes Ingress (and Service) resources and automatically creates/updates DNS records in AWS Route53. When you annotate an Ingress with `external-dns.alpha.kubernetes.io/hostname: app.example.com`, ExternalDNS creates an A record (or CNAME) in Route53 pointing to the ALB. No more manual DNS management.

Exercise 8.1: Deploy ExternalDNS

- ▶ Add **ExternalDNS** as a release in your `helmfile.yaml`
- ▶ Use the Bitnami or official ExternalDNS Helm chart
- ▶ Configure it for **AWS Route53** as the DNS provider
- ▶ Set `policy: sync` (manages full lifecycle — creates AND deletes records)
- ▶ Set `txtOwnerId` to a unique identifier for your cluster
- ▶ Use **IRSA** for Route53 permissions (`route53:ChangeResourceRecordSets` and `route53:ListHostedZones`)
- ▶ Limit ExternalDNS to your specific hosted zone using `domainFilters`
- ▶ Deploy to namespace `kube-system` or `external-dns`

Exercise 8.2: Configure Ingress Annotations for DNS

- ▶ Update your **frontend** Ingress to include the ExternalDNS hostname annotation
- ▶ Update your **backend** Ingress similarly (e.g., `api.example.com`)
- ▶ Deploy and verify:
 - ▶ • Check ExternalDNS logs: `kubectl logs -f deploy/external-dns`
 - ▶ • Verify Route53 records were created in the AWS Console
 - ▶ • DNS resolution: `dig app.example.com` should return the ALB address
 - ▶ • Access your app via the domain name in a browser

Warning

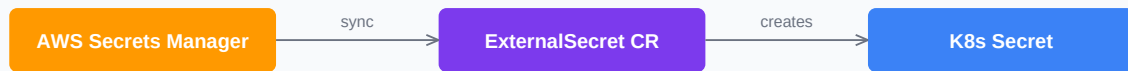
If you use `policy: sync`, ExternalDNS will **delete** DNS records when the corresponding Ingress is removed. This is the correct behavior for automation, but be aware of it. Use `policy: upsert-only` if you want ExternalDNS to only create/update but never delete records.

9

ExternalSecrets

AWS Secrets Manager to K8s Secrets

ExternalSecrets Operator Flow



Secrets stay in AWS, ESO syncs them into K8s -- no secrets in Git

The **ExternalSecrets Operator (ESO)** syncs secrets from external secret stores (AWS Secrets Manager, SSM Parameter Store, Vault, etc.) into Kubernetes Secrets. This means you never store secret values in Git, Helm values files, or anywhere in your repo. You define *what* to sync using an ExternalSecret custom resource, and ESO handles the rest.

How It Works

- **SecretStore / ClusterSecretStore** — defines the connection to AWS Secrets Manager (auth via IRSA)
- **ExternalSecret** — declares which secret keys to fetch and what K8s Secret to create
- ESO periodically syncs (default: every hour) and creates/updates the K8s Secret
- Your pods reference the K8s Secret normally via `envFrom` or `secretKeyRef`

Exercise 9.1: Deploy ExternalSecrets Operator

- ▶ Add the **external-secrets** Helm chart as a release in `helmfile.yaml`
- ▶ Deploy to namespace `external-secrets`
- ▶ Create an **IAM role** with `secretsmanager:GetSecretValue` and `secretsmanager:DescribeSecret` permissions
- ▶ Associate the role with ESO's ServiceAccount via **IRSA**
- ▶ Verify: `kubectl get pods -n external-secrets` — the operator should be running

Exercise 9.2: Create Secrets in AWS and Sync Them

- ▶ In AWS Secrets Manager, create a secret (e.g., `my-app/production/db-credentials`) containing your database credentials as a JSON object with keys for `username`, `password`, `host`, and `port`
- ▶ Create a **ClusterSecretStore** (or namespace-scoped **SecretStore**) pointing to AWS Secrets Manager in your region
- ▶ Create an **ExternalSecret** that:
 - ▶ • References the **ClusterSecretStore**
 - ▶ • Fetches `my-app/production/db-credentials`

- ▶ • Creates a K8s Secret named db-credentials
- ▶ • Maps individual keys (username, password, host, port)
- ▶ • Sets refreshInterval: 1h
- ▶ Verify: `kubectl get externalsecret` should show status SecretSynced
- ▶ Verify: `kubectl get secret db-credentials -o yaml` should exist with the correct data

Exercise 9.3: Wire Secrets into the Backend

- ▶ Update your **backend** Deployment to use the synced Secret for database connection:
 - ▶ • DB_HOST, DB_PORT from secretKeyRef
 - ▶ • DB_USER, DB_PASSWORD from secretKeyRef
- ▶ Verify the backend connects to the database successfully using the synced credentials
- ▶ Test secret rotation: update the secret in AWS Secrets Manager, wait for sync, verify the new value appears in K8s

Best Practice

Use **ClusterSecretStore** if multiple namespaces need access to the same AWS account secrets. Use namespace-scoped **SecretStore** for stricter isolation (e.g., dev team can only access dev secrets).

Karpenter vs Cluster Autoscaler

Karpenter (Recommended)

Pending Pod detected

Provisions optimal node

Directly provisions EC2
Faster, smarter bin-packing

Cluster Autoscaler

Pending Pod detected

Scales ASG up

Works through ASGs
Slower, less flexible

Karpenter (Recommended)

Karpenter is an open-source Kubernetes node provisioner built by AWS. Unlike Cluster Autoscaler which works through Auto Scaling Groups, Karpenter directly provisions EC2 instances based on the exact requirements of pending pods. It's faster (typically 60-90 seconds vs 3-5 minutes for CA) and smarter about bin-packing — it selects the optimal instance type from dozens of options.

Why Karpenter Over Cluster Autoscaler?

- **Speed** — provisions nodes in ~60 seconds vs 3-5 minutes
- **Flexibility** — chooses from many instance types/sizes based on pod requirements
- **Cost optimization** — better bin-packing, automatic spot/on-demand selection
- **Consolidation** — actively moves pods and removes underutilized nodes
- **Simpler** — no ASG management, fewer moving parts

If you need to use **Cluster Autoscaler** instead (e.g., non-EKS or specific organizational requirements), exercise 10.3 covers that as an alternative.

Exercise 10.1: Deploy Karpenter

- ▶ Add **Karpenter** as a release in your `helmfile.yaml`
- ▶ Karpenter needs:
 - ▶ • An **IAM role** for the Karpenter controller (IRSA)
 - ▶ • An **instance profile** for the nodes it creates
 - ▶ • SQS queue for spot interruption handling
- ▶ Deploy to namespace `kube-system` or `karpenter`
- ▶ Verify: `kubectl get pods -n karpenter` — the controller should be running

Hint: Follow the official Karpenter getting-started guide for EKS — the IAM setup has many moving parts.

Exercise 10.2: Configure NodePool and EC2NodeClass

- ▶ Create a **NodePool** (Karpenter v1) for **app workloads**:
 - ▶ • Instance categories: c, m, r families
 - ▶ • Allow spot and on-demand
 - ▶ • Set limits: max 20 CPU, 80Gi memory
 - ▶ • Labels: role=app
 - ▶ • Enable consolidationPolicy: WhenEmptyOrUnderutilized
- ▶ Create a **NodePool** for **database workloads**:
 - ▶ • Instance categories: r (memory-optimized) only
 - ▶ • On-demand only (no spot for databases)
 - ▶ • Labels: role=db
 - ▶ • Taints: workload=database:NoSchedule
- ▶ Create an **EC2NodeClass** defining subnet, security group, and AMI selectors
- ▶ Test: scale your frontend to 20 replicas and watch Karpenter provision new nodes
- ▶ Verify: `kubectl get nodeclaim` and `kubectl get nodes`

Exercise 10.3: Alternative: Cluster Autoscaler

- ▶ *(Do this only if you're not using Karpenter)*
- ▶ Deploy **Cluster Autoscaler** via Helmfile
- ▶ Configure it to discover ASGs by tags (`k8s.io/cluster-autoscaler/enabled`)
- ▶ Set `scale-down-delay-after-add: 10m` and `scale-down-unneeded-time: 10m`
- ▶ Use IRSA for the autoscaling IAM permissions
- ▶ Test by scaling up a deployment beyond current node capacity

11

Final Integration Exercise

Deploy everything end-to-end

This is the capstone exercise. You will deploy the entire stack from scratch using a single `helmfile sync` command and verify every component is working.

Exercise 11.1: Full Stack Deployment

- ▶ **Starting from a clean EKS cluster**, deploy everything with one command: `helmfile sync`
- ▶ Your `helmfile.yaml` should include ALL of these releases in the correct dependency order:

#	Release	Namespace	Depends On
1	kube-prometheus-stack	monitoring	—
2	external-secrets	external-secrets	—
3	external-dns	external-dns	—
4	karpenter	karpenter	—
5	database	production	external-secrets
6	backend	production	database
7	frontend	production	backend

Exercise 11.2: Verification Checklist

- ▶ After `helmfile sync` completes, verify each component:

Pods & Scheduling

- ▶ All pods are Running: `kubectl get pods -A`
- ▶ Frontend and backend are on app-nodes: `kubectl get pods -n production -o wide`
- ▶ Database is on a db-node

Networking & DNS

- ▶ Frontend accessible via domain name in browser
- ▶ Backend API accessible via its domain name
- ▶ Route53 records exist and resolve correctly

Monitoring

- ▶ Prometheus targets show all three ServiceMonitors
- ▶ Grafana dashboards display real data
- ▶ At least one alert rule is configured

Autoscaling

- ▶ HPA is active for frontend and backend: `kubectl get hpa -n production`
- ▶ VPA recommendations are available: `kubectl describe vpa -n production`
- ▶ Load test triggers HPA scale-up which triggers Karpenter node provisioning

Secrets

- ▶ ExternalSecrets show SecretSynced status
- ▶ Backend connects to DB using synced credentials
- ▶ No secrets in Git or Helm values files

Backups

- ▶ Manual backup job succeeds
- ▶ Backup file exists in S3
- ▶ Restore to a test instance works

Exercise 11.3: Chaos Testing (Bonus)

- ▶ Kill a frontend pod — does HPA replace it? Does the app stay available?
- ▶ Drain an app node — do pods reschedule? Does Karpenter handle it?
- ▶ Delete the K8s Secret — does ExternalSecrets re-create it?
- ▶ Simulate high load — does HPA scale up? Does Karpenter provision nodes?
- ▶ Delete the database pod — does the StatefulSet recreate it with data intact (PVC)?



Appendix

Useful Commands Quick Reference

This appendix lists the key commands you will use throughout the exercises, grouped by category. You should be comfortable looking up full syntax and flags from the official documentation.

Helmfile Commands

Command	Purpose
<code>helmfile lint</code>	Validate all charts
<code>helmfile diff</code>	Preview changes before deploying
<code>helmfile sync</code>	Deploy or upgrade all releases
<code>helmfile apply</code>	Diff + sync in one step
<code>helmfile destroy</code>	Remove all releases
<code>helmfile status</code>	Show status of all releases
<code>helmfile template</code>	Render all templates locally
<code>helmfile -e <env> sync</code>	Deploy with environment overrides

Key Debugging Areas

You should know how to: list all pods across namespaces with node placement, describe a pod to see scheduling decisions and events, follow container logs, view recent cluster events sorted by time, check resource usage with `kubectl top`, inspect HPA and VPA status, check ExternalSecret sync status, list Karpenter node claims, and follow Karpenter controller logs.

Monitoring Access

To access Grafana, Prometheus, and Alertmanager, use `kubectl port-forward` against the respective services in the monitoring namespace. Default ports: Grafana on 3000 (forwarding from port 80), Prometheus on 9090, Alertmanager on 9093.

Backup Operations

To trigger a manual backup, create a one-off Job from your CronJob resource. Check job completion status with `kubectl`. List your S3 backups using the AWS CLI against your backup bucket path.

AWS / EKS Essentials

Key operations: update your kubeconfig for the cluster, list node groups, verify IRSA annotations on ServiceAccounts (look for the role-arn annotation), and verify ExternalDNS Route53 records using the AWS CLI.

Good luck — build it, break it, fix it. That's how you learn production Kubernetes.